

SalesVerse 2.0

Code Standards and Quality Report

Coding standards, static analysis, tests, coverage, dependencies and licences

Prepared for	Insurans Islam Family Takaful Sendirian Berhad
Solution	IIFT Agent/Banca Portal & Back-office
Build assessed	Commit 47f239c
Evidence date	2 October 2026, 14:55 UTC
Document date	October 2026
Version	1.0
Classification	Commercial-in-Confidence

Commercial-in-Confidence. Prepared by iorta TechNXT for Insurans Islam Family Takaful Sendirian Berhad as part of the proposal for the Agent/Banca Portal and Back-office Solution.

Contents

Contents.....	2
1. Summary.....	3
2. Coding standards.....	4
CI pipeline gates.....	4
3. Static analysis.....	5
4. Automated tests.....	6
What the suites cover.....	6
5. Coverage.....	8
Coverage targets for implementation.....	8
6. Dependencies and licences.....	10
Vulnerability audit.....	10
Licence inventory.....	10
7. Code metrics.....	10
8. Maintainability.....	11
9. Known limitations and improvement plan.....	12
Annex A – Test titles.....	13
Annex B – Evidence files.....	17

1. Summary

This report describes the coding standards used to build SalesVerse 2.0 and the quality evidence for the current build (commit 47f239c). All results are produced by tools, not by assessment, and the report is rebuilt from the evidence files each time they are refreshed.

Measure	Result
Lint (oxlint)	0 errors, 0 warnings
Type check (TypeScript compiler)	0 errors, 0 warnings
Formatting (Prettier)	0 errors, 0 warnings
API unit tests	59 of 59 passed
API end-to-end tests	32 of 32 passed
Web unit and component tests	81 of 81 passed
End-to-end line coverage (API)	68.06% of 131 source files
Unit line coverage (API)	14.89% of 131 source files
Dependency vulnerabilities (npm audit)	0 found
Runtime packages in licence inventory	494
Web lint (oxlint)	0 errors, 0 warnings (184 files, 213 rules)
Web type check (tsc)	No errors reported
Web production build (Vite)	No errors reported; largest chunk antd 1,378.82 kB (437.98 kB gzip)

Table 1: Quality results

Static analysis, the three test suites and the dependency audit are clean. Coverage is measured over all 131 API source files, including files that no test loads. End-to-end coverage is 68.06% of lines because those tests drive the real HTTP pipeline against PostgreSQL. Unit coverage is lower by design at this stage: unit tests target the calculation and rule code where defects are most costly. Section 5 sets the coverage targets for implementation.

Coding standards

The standards below apply to every change. Most are enforced by tools in the CI pipeline, so a change that breaks them cannot be merged.

Area	Standard	Enforced by
Language	TypeScript with strict, noImplicitReturns, noFallthroughCasesInSwitch, isolatedModules; target ES2023; module nodenext (ES modules on Node.js 22)	Type check in CI
Module structure	One NestJS module per functional area (22 modules: access, agency, aml, audit, auth, billing, claims, dashboard, documents, eod, esign, health, integration, integration-api, issues, notifications, participants, policies, products, reports, settings, workflow); shared code in src/common	Code review
Naming	Files in kebab-case with role suffix (.controller, .service, .dto, .guard, .spec); classes in PascalCase; database tables and columns in snake_case mapped from camelCase	Code review
Input validation	Every request body is a DTO class with class-validator rules; the global pipe uses whitelist, forbidNonWhitelisted and transform, so unknown fields are rejected	Global ValidationPipe; e2e tests
Data access	Prisma client only. 9 raw SQL statements use Prisma's tagged templates, which bind parameters; 0 use the unsafe string API	Code review; source scan in this report
Error handling	Business errors carry a code and a user message; one global filter returns status, code, message and correlation id, and logs full detail only on the server for 5xx	AllExceptionsFilter; security checks
Logging	Structured JSON logs (pino) with the request correlation id; cookies, authorisation and CSRF headers redacted; console logging forbidden	no-console lint rule
Comments	Comments explain why, not what; public services and guards carry a short doc comment, often citing the RFP requirement (for example AP-01/03/04)	Code review
Formatting	Prettier with single quotes, trailing commas 'all' and a line width of 100	prettier --check in CI
Linting (API)	eslint plugins typescript, unicorn, oxc, import; categories correctness=error, suspicious=warn; rules as errors: no-console, typescript/no-explicit-any, eqeqeq, no-eval	Lint step in CI
Linting (web)	eslint plugins typescript, react, react-perf, jsx-a11y, unicorn, oxc, import, including accessibility (jsx-a11y) and React hooks rules	Lint step in CI
Commits and review	Small commits with a descriptive message; every change through a pull request reviewed by a second engineer; the reviewer checks security, tests and RFP traceability	Pull-request review; branch protection on main
Branching	Short-lived feature branches from main; main is always releasable; releases are tagged	Branch protection

Table 2: Coding standards

CI pipeline gates

The pipeline runs on every pull request and on main. A failing step stops the merge.

Job	Step	Command or action
secrets	Secret scan (full history)	docker run --rm -v "\$PWD:/repo" zricethezav/gitleaks:v8.21.2 detect...
codeql	CodeQL set-up: javascript-typescript, security-extended	github/codeql-action/init@v3
codeql	CodeQL analysis (SAST)	github/codeql-action/analyze@v3
verify	npm ci	npm ci
verify	Generate database client	npx prisma generate
verify	Formatting	npx prettier --check "apps/*/src/**/*.{ts,tsx}" "apps/api/test/**/*..."
verify	Lint	npm run lint
verify	Type check	npm run typecheck --workspaces

Job	Step	Command or action
verify	Unit tests	npm test --workspaces
verify	End-to-end API tests	npm run test:e2e --workspace apps/api
verify	Build	npm run build
verify	Dependency audit	npm audit --audit-level=moderate
verify	Software bill of materials	npm sbom --sbom-format cyclonedx --omit dev > sbom.cdx.json
verify	Keep sbom.cdx.json as build artifact	actions/upload-artifact@v4
images	Build image from apps/api/Dockerfile	docker build -f apps/api/Dockerfile --target runtime -t salesverse-...
images	Build image from apps/web/Dockerfile	docker build -f apps/web/Dockerfile -t salesverse-web:\${{ github.sh...
images	Container vulnerability scan	aquasecurity/trivy-action@0.28.0

Table 3: CI steps (.github/workflows/ci.yml)

3. Static analysis

Run on 2 October 2026, 14:57 UTC with the same commands as the CI pipeline.

Check	Tool	Command	Errors	Warnings	Result
Lint	oxlint 1.86.0	npx oxlint src test	0	0	Pass
Type check	TypeScript compiler 6.0.3	npx tsc --noEmit -p tsconfig.json	0	0	Pass
Formatting	Prettier 3.9.9	npx prettier --check apps/api/src/**/*.ts apps/api/test/**/*.ts apps/api/prisma/**/*.ts	0	0	Pass

Table 4: API static analysis

Web application check	Result
Web lint (oxlint)	0 errors, 0 warnings (184 files, 213 rules)
Web type check (tsc)	No errors reported
Web production build (Vite)	No errors reported; largest chunk antd 1,378.82 kB (437.98 kB gzip)

Table 5: Web application static analysis and build

4. Automated tests

Unit tests run with Vitest and need no database. End-to-end tests start the API with the production HTTP pipeline (the same middleware, guards, validation and error handling as the server) against a PostgreSQL 16 database, and call it over HTTP. Web unit and component tests run with Vitest and Testing Library in a simulated browser (jsdom) and cover the API client, formatting, permissions, navigation and shared components. All three suites run in CI on every change.

Suite	Files	Tests	Passed	Failed	Run
API unit tests	13	59	59	0	2 October 2026, 14:55 UTC
API end-to-end tests	4	32	32	0	2 October 2026, 14:55 UTC
Web unit and component tests	16	81	81	0	2 October 2026, 14:56 UTC

Table 6: Test results

What the suites cover

Test file	What it asserts	Tests	Passed
app-config.spec.ts	Start-up configuration: safe defaults, weak secrets rejected, insecure production settings refused	4	4
field-crypto.service.spec.ts	AES-256-GCM field encryption, tamper detection, blind index for identifiers	4	4
dates.spec.ts	Age at next birthday and Brunei business-day boundaries (UTC+8)	4	4
name-matching.spec.ts	AML fuzzy name matching: normalisation, honorifics, transliterations, thresholds	5	5
watchlist.spec.ts	Watch-list file parsing (quoted fields, CRLF)	1	1
password.service.spec.ts	Password policy rules, username check, history and expiry	6	6
file-inspector.spec.ts	Upload checks by content, forced extension, size limit	4	4
plan-label.spec.ts	Configured plan names on documents and the e-signature page, with fallback to the code	3	3
product-definitions.spec.ts	Product questionnaires: referral answers, completeness, unknown questions	4	4
reports.spec.ts	CSV formula neutralisation and report schedules in Brunei time	4	4
workflow-payload.spec.ts	Encrypted values removed from approval request payloads before they are shown	2	2
financing.engine.spec.ts	Mortgage Takaful rating (FTP-HP, FTP-NP, PFT) and the B\$150,000 referral	7	7
fixed-plan.engine.spec.ts	Annual plan rating (PHA, PRO, KHR, OSA), eligibility and riders	11	11
access-control.e2e-spec.ts	Authentication, audience separation, permissions and data scoping over HTTP	11	11
auth.e2e-spec.ts	Sign-in, cookie flags, logout, password change, logout and session revocation	7	7
operations.e2e-spec.ts	Onboarding, AML review, issues with SLA and other back-office journeys	10	10
policy-lifecycle.e2e-spec.ts	Quotation to policy with maker-checker payment, agency block, referral approval	4	4
client.test.ts	API client: base path, CSRF header, uploads, error mapping, session-ended handling	12	12
ErrorAlert.test.tsx	Error messages, validation details and the support reference for server errors	4	4
Money.test.tsx	Amount display	2	2
RequirePermission.test.tsx	Pages shown only to users holding the permission	2	2

Test file	What it asserts	Tests	Passed
StatusTag.test.tsx	Status labels and colours	3	3
menus.test.ts	Menu highlighting for the current page	6	6
format.test.ts	Money, number, date and file-size formatting in Brunei dollars and local time	11	11
permissions.test.ts	Permission codes unique, prefixed by audience, approval permissions grouped	3	3
status.test.ts	Status colour mapping	2	2
LoginPage.test.tsx	Sign-in form: validation, forced password change, rejected credentials, ended session	6	6
CellText.test.tsx	Table cell text	2	2
links.test.ts	Links to record pages for each audience and from notifications	5	5
values.test.ts	Display of approval values: hidden encrypted and hashed keys, labels, money and dates	8	8
NomineesEditor.test.ts	Nominee rows: existing ids kept, blank ID number keeps the stored one, shares	8	8
PaymentList.test.tsx	Payment list filtered by the status in the URL	3	3
options.test.ts	Option labels, policy reference and term display	4	4

Table 7: Test files

Every test title is listed in Annex A.

5. Coverage

Coverage is measured with the V8 provider over all 131 API source files under apps/api/src, excluding the Prisma database client in src/generated, the test files and the start-up file main.ts. Files that a suite never loads count as uncovered. Earlier versions of this report measured end-to-end coverage over the files the tests loaded, so the figures here are lower but complete. Front-end coverage is not yet measured.

Measure	Unit tests	End-to-end tests
Lines	14.89%	68.06%
Statements	15.29%	68.27%
Functions	11.25%	70.52%
Branches	18.68%	53.8%

Table 8: Overall coverage

Module or area	Lines	Unit	End-to-end
access	139	0.0%	58.3%
agency	230	0.0%	38.3%
aml	197	36.5%	77.2%
app.module.ts	3	0.0%	66.7%
app.setup.ts	18	0.0%	94.4%
audit	21	4.8%	90.5%
auth	212	14.6%	83.0%
billing	144	0.0%	53.5%
claims	71	0.0%	73.2%
common	172	32.6%	83.1%
config	81	86.4%	66.7%
dashboard	45	0.0%	91.1%
documents	223	9.9%	74.4%
eod	80	0.0%	83.8%
esign	78	0.0%	42.3%
health	19	0.0%	36.8%
integration	131	0.0%	75.6%
integration-api	55	0.0%	49.1%
issues	111	0.0%	69.4%
notifications	84	0.0%	57.1%
participants	83	0.0%	49.4%
policies	378	0.0%	63.2%
products	68	48.5%	67.6%
products/rating (rating engines)	193	83.9%	71.5%
reports	232	15.1%	84.5%
settings	85	4.7%	65.9%
workflow	163	4.9%	70.6%

Table 9: Line coverage by module

Coverage targets for implementation

Coverage is reported honestly rather than inflated with low-value tests. The rating engines already carry focused unit tests; most services are covered only through end-to-end journeys. During implementation we add service-level unit tests and SIT automation, with these targets:

Area	Target	Measured by
Rating engines (products/rating)	At least 80% lines and branches	Unit tests, per release
Billing (payments, grace period)	At least 80% lines	Unit plus end-to-end
Workflow (maker-checker)	At least 80% lines	Unit plus end-to-end

Area	Target	Measured by
All API code	At least 70% lines combined	Merged unit and end-to-end coverage in CI

6. Dependencies and licences

Vulnerability audit

npm audit reports 0 known vulnerabilities across 933 installed packages (514 production, 405 development, 136 optional). CI fails on any moderate or higher finding, and a container scan (Trivy) fails the image build on any fixable Critical or High finding. Each CI run also produces a CycloneDX software bill of materials of the production dependencies (npm sbom), kept as a build artifact.

Severity	Critical	High	Moderate	Low	Info	Total
Findings	0	0	0	0	0	0

Licence inventory

The runtime dependency tree contains 494 packages. All carry permissive open-source licences apart from the items explained below; none imposes copyleft obligations on the deployed solution.

Licence	Packages
MIT	400
ISC	40
Apache-2.0	30
BSD-3-Clause	7
Unlicense	3
UNKNOWN	2
MIT/X11	2
BSD-2-Clause	2
MIT and ISC	1
Python-2.0	1
EPL-2.0	1
(MIT OR GPL-3.0-or-later)	1
MIT-0	1
(MIT AND Zlib)	1
0BSD	1
MIT AND ISC	1

Table 10: Packages by licence

Package	Version	Declared licence	Position
buffers	0.1.1	UNKNOWN	MIT licence declared in the project repository but not in package.json.
elkjs	0.11.1	EPL-2.0	Reached only through the Prisma CLI's optional Studio peer, a development tool. It is not installed in the runtime image, which is built with <code>npm ci --omit=dev</code> .
jszip	3.10.2	(MIT OR GPL-3.0-or-later)	Dual-licensed MIT OR GPL-3.0; used under the MIT licence.
png-js	2.0.0	UNKNOWN	MIT licence declared in the project repository but not in package.json.

Table 11: Licence exceptions and how they are handled

7. Code metrics

Measure	Value
API source files (including unit test files)	145
API source lines (code / comment / blank)	15,220 / 549 / 1,169

Measure	Value
End-to-end test files and lines of code	7 files, 988 lines
API unit test files	13
Web unit and component test files	16
Seed and reference data (files / lines of code)	6 / 1,818
Functional modules	22
API paths / operations	128 / 149
Database tables / enumerations	37 / 36
Schema lines / migration SQL lines	915 / 883

Table 12: Size metrics (API)

Generated code (the Prisma client in src/generated) is excluded from linting and from these counts.

8. Maintainability

Each functional area is a separate NestJS module with its own controller, services and DTOs. Modules talk through injected services, not shared tables, so an area can be changed, tested or later moved to its own service without touching the others.

Business behaviour that IIFT will want to change is configuration, not code. Products, plans and rate tables, questionnaires, workflow definitions and approval thresholds, security and business parameters (password policy, lockout, session limits, grace period, SLA hours) and master data are maintained in the back-office, audited and take effect without a release. Product and rate changes are checked by a second person until a maker-checker approval type for products is added during implementation.

The API, the web tier and the database schema are versioned together. Database changes are applied by versioned SQL migrations, and the same container images move from SIT to UAT to production.

9. Known limitations and improvement plan

Area	Current position	Action during implementation	When
Service-level unit coverage	Unit tests concentrate on rules and calculations; most services are exercised through the end-to-end suite only.	Add unit and SIT automation; reach at least 80% line coverage on rating, billing and workflow modules	Weeks 6–19
Front-end tests	Unit and component tests cover the API client, formatting, permissions, navigation and shared components; complete pages are not yet tested in a browser.	Add Playwright smoke tests of the main portal and back-office journeys to CI	Weeks 6–16
Front-end bundle size	The Ant Design vendor chunk is about 1.38 MB raw (438 kB gzip). It is cached long-term and pages are already split by route, so it is loaded once per release.	Watch the size at each release; import components more selectively if first-load time on branch networks approaches the NFR-01 target	Each release
Integration adapters	Core, FIN, AML provider and SMS adapters run in simulated mode until IIFT endpoints are available.	Contract tests per interface specification; joint tests with system owners	Weeks 8–17
Performance evidence	No load test results yet.	k6 load, stress and soak tests on production-like SIT (DEL-18)	Weeks 17–18
Mutation and accessibility checks	Not yet part of CI.	Add an accessibility check on key screens and a periodic mutation-test run on rating engines	Weeks 10–19

Table 13: Improvement plan

Annex A – Test titles

Test file	Test	Result
app-config.spec.ts	loads a minimal development configuration with safe defaults	Passed
app-config.spec.ts	fails fast on missing or weak secrets	Passed
app-config.spec.ts	refuses insecure production settings	Passed
app-config.spec.ts	accepts a complete production configuration	Passed
field-crypto.service.spec.ts	round-trips a value and never produces the same ciphertext twice	Passed
field-crypto.service.spec.ts	detects tampering through the authentication tag	Passed
field-crypto.service.spec.ts	produces the same blind index for differently formatted identifiers	Passed
field-crypto.service.spec.ts	normalises and masks identifiers	Passed
dates.spec.ts	computes age at next birthday	Passed
dates.spec.ts	uses the Brunei calendar day (UTC+8) as the business date	Passed
dates.spec.ts	returns the UTC instants bounding a Brunei business day	Passed
dates.spec.ts	adds months and counts days	Passed
name-matching.spec.ts	normalises case, punctuation, honorifics and common spellings	Passed
name-matching.spec.ts	scores identical names 100 regardless of honorific	Passed
name-matching.spec.ts	scores close transliterations above the default review threshold	Passed
name-matching.spec.ts	scores unrelated names low	Passed
name-matching.spec.ts	implements Jaro-Winkler	Passed
watchlist.spec.ts	handles quoted fields, escaped quotes and CRLF line endings	Passed
password.service.spec.ts	accepts a password meeting every rule	Passed
password.service.spec.ts	reports every unmet rule together	Passed
password.service.spec.ts	rejects passwords containing the username	Passed
password.service.spec.ts	hashes with Argon2id and verifies	Passed
password.service.spec.ts	expires passwords after the configured number of days	Passed
password.service.spec.ts	generates temporary passwords that satisfy the policy	Passed
file-inspector.spec.ts	identifies files by content and forces a matching extension	Passed
file-inspector.spec.ts	rejects content that is not PDF, PNG or JPEG whatever its name	Passed
file-inspector.spec.ts	enforces the size limit and requires a file	Passed
file-inspector.spec.ts	builds safe file names	Passed
plan-label.spec.ts	uses the configured names	Passed
plan-label.spec.ts	falls back to the code for a retired plan	Passed
plan-label.spec.ts	returns null when the product has no plans	Passed
product-definitions.spec.ts	returns referral reasons for "yes" answers that require underwriting	Passed
product-definitions.spec.ts	requires every question to be answered and details for "yes"	Passed
product-definitions.spec.ts	rejects answers to questions the product does not ask	Passed
product-definitions.spec.ts	treats documents as mandatory unless stated otherwise	Passed
reports.spec.ts	quotes CSV cells and neutralises spreadsheet formulas	Passed
reports.spec.ts	runs daily reports at 07:00 Brunei time (23:00 UTC)	Passed
reports.spec.ts	runs weekly reports on Monday and monthly reports on the 1st (local time)	Passed
reports.spec.ts	resolves relative periods to date ranges	Passed
workflow-payload.spec.ts	removes encrypted values at any depth and keeps everything else	Passed
workflow-payload.spec.ts	leaves scalar and empty payloads untouched	Passed
financing.engine.spec.ts	calculates a decreasing-term contribution on the flat-rate financing total	Passed
financing.engine.spec.ts	refers financing above the B\$150,000 high-risk limit to IIFT Sales	Passed
financing.engine.spec.ts	does not refer financing exactly at the high-risk limit	Passed
financing.engine.spec.ts	rejects cover that would run past the maximum age at expiry	Passed
financing.engine.spec.ts	applies no profit loading when the product uses NONE as profit basis	Passed
financing.engine.spec.ts	lists every missing risk field at once	Passed
financing.engine.spec.ts	validates its configuration	Passed
fixed-plan.engine.spec.ts	prices Professional Plan B with the optional additional cover (FFR03)	Passed
fixed-plan.engine.spec.ts	accepts only Occupational Class I for the Professional plan	Passed
fixed-plan.engine.spec.ts	applies the Wider (Child) loading for Khairat (FFR04)	Passed
fixed-plan.engine.spec.ts	requires a coverage type when the product defines them	Passed
fixed-plan.engine.spec.ts	offers 1- and 2-year cover for Personal Home Assistant (FFR02)	Passed
fixed-plan.engine.spec.ts	accepts a registered Brunei citizen student	Passed
fixed-plan.engine.spec.ts	rejects a participant who is not a Brunei citizen	Passed

Test file	Test	Result
fixed-plan.engine.spec.ts	requires confirmation of student registration	Passed
fixed-plan.engine.spec.ts	rejects participants older than 65 next birthday	Passed
fixed-plan.engine.spec.ts	drops risk details that the product does not define	Passed
fixed-plan.engine.spec.ts	validates every catalogue product configuration	Passed
access-control.e2e-spec.ts	rejects unauthenticated requests	Passed
access-control.e2e-spec.ts	keeps portal users out of the back-office API and vice versa	Passed
access-control.e2e-spec.ts	enforces permissions within the back-office	Passed
access-control.e2e-spec.ts	restricts records to the agent hierarchy and agency	Passed
access-control.e2e-spec.ts	treats injection attempts in search parameters as plain text	Passed
access-control.e2e-spec.ts	rejects unknown and malformed fields	Passed
access-control.e2e-spec.ts	never exposes internal error details and returns a correlation id	Passed
access-control.e2e-spec.ts	sends security headers and hides the framework	Passed
access-control.e2e-spec.ts	authenticates system-to-system calls with an API key	Passed
access-control.e2e-spec.ts	stores identification numbers encrypted and shows them masked	Passed
access-control.e2e-spec.ts	keeps the audit trail append-only at database level	Passed
auth.e2e-spec.ts	rejects wrong credentials with a generic message	Passed
auth.e2e-spec.ts	issues an HttpOnly, SameSite=Strict session cookie and never returns secrets	Passed
auth.e2e-spec.ts	locks the account after the configured number of failed attempts	Passed
auth.e2e-spec.ts	requires the CSRF token on state-changing requests	Passed
auth.e2e-spec.ts	ends the session on logout	Passed
auth.e2e-spec.ts	keeps only one active session per user	Passed
auth.e2e-spec.ts	forces a temporary password to be changed before anything else	Passed
operations.e2e-spec.ts	onboards a sub-agent through registration, AML, documents and approval (AP-07, BO-10/13)	Passed
operations.e2e-spec.ts	asks passport holders for a passport copy instead of an IC copy	Passed
operations.e2e-spec.ts	closes a pending registration when Compliance confirms an AML match	Passed
operations.e2e-spec.ts	routes a watch-list match to Compliance and records the review (AP-16, BO-13..15)	Passed
operations.e2e-spec.ts	manages an issue through assignment, comments and resolution with SLA dates (AP-55..57, BO-29..31)	Passed
operations.e2e-spec.ts	accepts claim notifications only within the period of cover (AP-43)	Passed
operations.e2e-spec.ts	runs every standard report and exports it in each format (BO-22..24)	Passed
operations.e2e-spec.ts	runs end-of-day, posts to FIN, reconciles and delivers integration messages (FFR EOD, INT-13..15)	Passed
operations.e2e-spec.ts	administers users, roles, parameters, master data and workflows with an audit trail (BO-03/04/26..33, COM-04)	Passed
operations.e2e-spec.ts	serves the dashboards (AP-52/53, BO-20/21)	Passed
policy-lifecycle.e2e-spec.ts	runs the full Professional Takaful Plan journey with maker-checker payment verification	Passed
policy-lifecycle.e2e-spec.ts	blocks new business for an agency with overdue contributions (AP-42)	Passed
policy-lifecycle.e2e-spec.ts	refers high-risk financing for approval and prevents self-approval	Passed
policy-lifecycle.e2e-spec.ts	renews a policy only inside its renewal window	Passed
client.test.ts	prefixes the API base path, drops empty query parameters and returns the JSON body	Passed
client.test.ts	sends the CSRF token and a JSON body on state-changing requests	Passed
client.test.ts	does not send the CSRF token on GET requests	Passed
client.test.ts	sends uploads as form data without a JSON content type	Passed
client.test.ts	resolves to undefined for 204 No Content	Passed
client.test.ts	turns an error response into an ApiError with details and correlation id	Passed
client.test.ts	uses a generic error when the response body is not JSON	Passed
client.test.ts	notifies session-ended listeners on 401	Passed
client.test.ts	does not end the session when the login itself is rejected	Passed
client.test.ts	stops notifying a listener after it unsubscribes	Passed
client.test.ts	does not end the session on other client errors	Passed
client.test.ts	raises an ApiError when the file cannot be downloaded	Passed
ErrorAlert.test.tsx	renders nothing when there is no error	Passed
ErrorAlert.test.tsx	shows the API message and each validation detail	Passed
ErrorAlert.test.tsx	shows the support reference for server errors only	Passed
ErrorAlert.test.tsx	falls back to a generic message for unexpected errors	Passed
Money.test.tsx	shows the formatted amount	Passed
Money.test.tsx	can be emphasised	Passed
RequirePermission.test.tsx	renders the page when the user holds the permission	Passed

Test file	Test	Result
RequirePermission.test.tsx	shows the not-authorised page when the permission is missing	Passed
StatusTag.test.tsx	shows the humanised status with its colour	Passed
StatusTag.test.tsx	prefers an explicit label	Passed
StatusTag.test.tsx	renders nothing without a status	Passed
menus.test.ts	highlights the dashboard only on the module root	Passed
menus.test.ts	highlights nothing on pages without a menu entry	Passed
menus.test.ts	highlights the entry whose path is a parent of the current page	Passed
menus.test.ts	prefers the longest matching entry	Passed
menus.test.ts	does not match entries that merely share a prefix	Passed
menus.test.ts	ignores a trailing slash	Passed
format.test.ts	formats amounts in Brunei dollars with two decimals and thousands separators	Passed
format.test.ts	rounds to the nearest cent and keeps zero as an amount	Passed
format.test.ts	shows a dash when there is no amount	Passed
format.test.ts	groups thousands and shows a dash when there is no value	Passed
format.test.ts	formats a calendar date as day, short month and year	Passed
format.test.ts	formats a local timestamp with a 24-hour time	Passed
format.test.ts	shows a dash for missing dates	Passed
format.test.ts	turns an enum code into sentence case	Passed
format.test.ts	shows a dash for an empty value	Passed
format.test.ts	splits camel-case keys into a sentence-case label	Passed
format.test.ts	uses bytes, kilobytes or megabytes depending on size	Passed
permissions.test.ts	are unique	Passed
permissions.test.ts	use the portal or back-office prefix matching their key	Passed
permissions.test.ts	group every approval permission and nothing else	Passed
status.test.ts	maps statuses to the tag colour of their stage	Passed
status.test.ts	falls back to the default colour for unknown or missing statuses	Passed
LoginPage.test.tsx	renders the username and password fields	Passed
LoginPage.test.tsx	asks for both fields when submitted empty	Passed
LoginPage.test.tsx	signs in with the typed credentials and opens the home page	Passed
LoginPage.test.tsx	sends users who must change their password to the change-password page	Passed
LoginPage.test.tsx	shows the error returned for rejected credentials	Passed
LoginPage.test.tsx	explains that the previous session ended	Passed
CellText.test.tsx	shows the text within the given width	Passed
CellText.test.tsx	shows a dash when there is no text	Passed
links.test.ts	links to the record page regardless of the entity type spelling	Passed
links.test.ts	uses the portal routes for portal users	Passed
links.test.ts	returns null when the audience has no page for the record or the id is missing	Passed
links.test.ts	keeps links that already point into a module	Passed
links.test.ts	prefixes audience-neutral links with the user module	Passed
values.test.ts	hides optimistic-lock versions and encrypted or hashed values only	Passed
values.test.ts	uses curated labels where needed and sentence case otherwise	Passed
values.test.ts	accepts plain objects only	Passed
values.test.ts	shows a dash for empty values and Yes/No for booleans	Passed
values.test.ts	formats numeric values of amount-like keys as money	Passed
values.test.ts	formats ISO dates and timestamps	Passed
values.test.ts	humanises enum codes that contain underscores or sit under an enum-like key	Passed
values.test.ts	serialises nested values as JSON	Passed
NomineesEditor.test.ts	starts with a single full-share nominee row when there are no nominees	Passed
NomineesEditor.test.ts	carries the id and masked ID number of existing nominees	Passed
NomineesEditor.test.ts	converts decimal-string shares to numbers	Passed
NomineesEditor.test.ts	returns no inputs when the form has no rows	Passed
NomineesEditor.test.ts	keeps the id of an existing nominee and trims the text fields	Passed
NomineesEditor.test.ts	drops a blank ID number so the stored one is kept	Passed
NomineesEditor.test.ts	sends new rows without an id and fills defaults for missing fields	Passed
NomineesEditor.test.ts	round-trips existing nominees without changing them	Passed
PaymentList.test.tsx	reads a known payment status	Passed
PaymentList.test.tsx	ignores missing or unknown statuses	Passed
PaymentList.test.tsx	starts filtered by the status in the URL	Passed

Test file	Test	Result
options.test.ts	label each code in sentence case	Passed
options.test.ts	uses the policy number once the policy is issued	Passed
options.test.ts	falls back to the quotation number before issuance	Passed
options.test.ts	shows whole years in years and anything else in months	Passed

Table 14: All automated tests

Annex B – Evidence files

Results in this report are read from the evidence files below when the report is built. The fingerprint is the start of each file's SHA-256 hash, so a reader can confirm which evidence a given version of the report was built from.

Evidence file	Size	SHA-256 (first 16 characters)
api-e2e-coverage-summary.json	42,784 bytes	74b2c247a495d346...
api-e2e-tests.json	895,673 bytes	bb1381c4fef07679...
api-static-analysis.json	796 bytes	38c06cf62d020977...
api-unit-coverage-summary.json	41,871 bytes	501662d3899973ad...
api-unit-tests.json	901,653 bytes	04b9d616f28c707b...
code-metrics-api.json	760 bytes	ec37c59d47c8708c...
npm-audit.json	366 bytes	1cf279e9ef691d09...
runtime-dependency-licences.json	40,391 bytes	b003a0dc1a75e06f...
security-checks.json	9,685 bytes	5ab9bdeea3c926fd...
web-build.txt	9,886 bytes	78590222981eafd9...
web-lint.json	197 bytes	21a8be1488eb17d7...
web-typecheck.txt	26 bytes	e5073c15448438bb...
web-unit-tests.json	24,628 bytes	7dac851fc00c909f...

Table 15: Evidence files